



Банківські транзакції — Посібник

користувача

Покроковий огляд застосунку «Banking Transactions» (Домашнє завдання 1) зі скріншотами: як його запустити, що він уміє та як виглядає кожна функція на практиці.

Проект — це повноцінна система банківських транзакцій, побудована **за принципом TDD**. Її ключова риса: **єдиний REST-контракт**, реалізований **чотирма взаємозамінними бекендами** та спожитий **трьома взаємозамінними фронтендами** — будь-яку пару можна поєднати. У цьому посібнику наживо показано по два з кожного.

1. Що всередині

Рівень	Варіанти	Розташування
Бекенди (обираєте один · усі на :3000 · однаковий контракт і тести)	Java + Spring Boot · Go + Chi · Python + FastAPI · C# / .NET	backend/ , backend-go/ , backend-fastapi/ , backend-dotnet/
Фронтенди (обираєте один)	Angular + Microsoft Fluent (веб) · Electron (десктоп) · Flutter Web	frontend/ , frontend-electron/ , frontend-flutter/

Можливості (реалізовані в кожному бекенді): створення / список / отримання транзакцій зі структурованими помилками валідації; фільтрація історії за рахунком, типом і діапазоном дат; **баланс**, **зведення** та **прості відсотки** за рахунком; **експорт у CSV**; **обмеження частоти запитів** за IP; інтерактивна API-документація та ендпоінт стану.

2. Як запустити

З теки homework-1/ запустіть інтерактивний лаунчер:

```
./run.sh
```

Оберіть **бекенд** (Java / Go / FastAPI / .NET) і **фронтенд** (Angular / Electron / Flutter). Далі він запускає бекенд на http://localhost:3000 , чекає готовності та **засіває демо-дані** (demo/seed.sh), запускає фронтенд і зупиняє все по **Ctrl+C**. Потрібні лише інструменти для обраного стека.

```
# Бекенд Python + FastAPI
cd backend-fastapi
python3 -m venv .venv && ./venv/bin/pip install -r requirements.txt
./venv/bin/uvicorn app.main:app --port 3000

# Бекенд Go + Chi
cd backend-go && go run ./cmd/server          # слухає :3000

# Фронтенд Angular (проксі /api -> :3000)
```

```
cd frontend && npm install && npm start      # http://localhost:4200

# Десктоп-фронтенд Electron
cd frontend-electron && npm install && npm start
```

Індикатор **API-online** у правому верхньому куті будь-якого фронтенду підтверджує зв'язок із бекендом.

3. Комбінування: два фронтенди × два бекенди (перевірено наживо)

Скріншоти в цьому посібнику зроблено на **двох різних стеках** — це доводить, що спільний контракт працює незалежно від комбінації:

Комбінація	Фронтенд	Бекенд
Стек А	Electron (десктоп)	Python · FastAPI
Стек В	Angular · Fluent (веб)	Go · Chi

 **One API contract · four backends · three frontends**
The launcher ./run.sh lets you mix any backend with any frontend. Below: two backends verified live, each serving the same seeded data on :3000.

BACKEND #1 Python · FastAPI frontend: Electron (desktop)

```
./run.sh # choose 3) FastAPI, then 2) Electron
venv + deps installed
Starting backend: Python + FastAPI
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:3000

curl -s :3000/openapi.json | jq .info
{ "title": "Banking Transactions API (FastAPI)",
  "version": "v1" }

# interactive Swagger UI -> :3000/docs
```

BACKEND #2 Go · Chi frontend: Angular (web :4200)

```
./run.sh # choose 2) Go, then 1) Angular
Starting backend: Go + Chi
2026/07/03 14:15:29 Banking API listening on :3000
Seeding sample data_ done

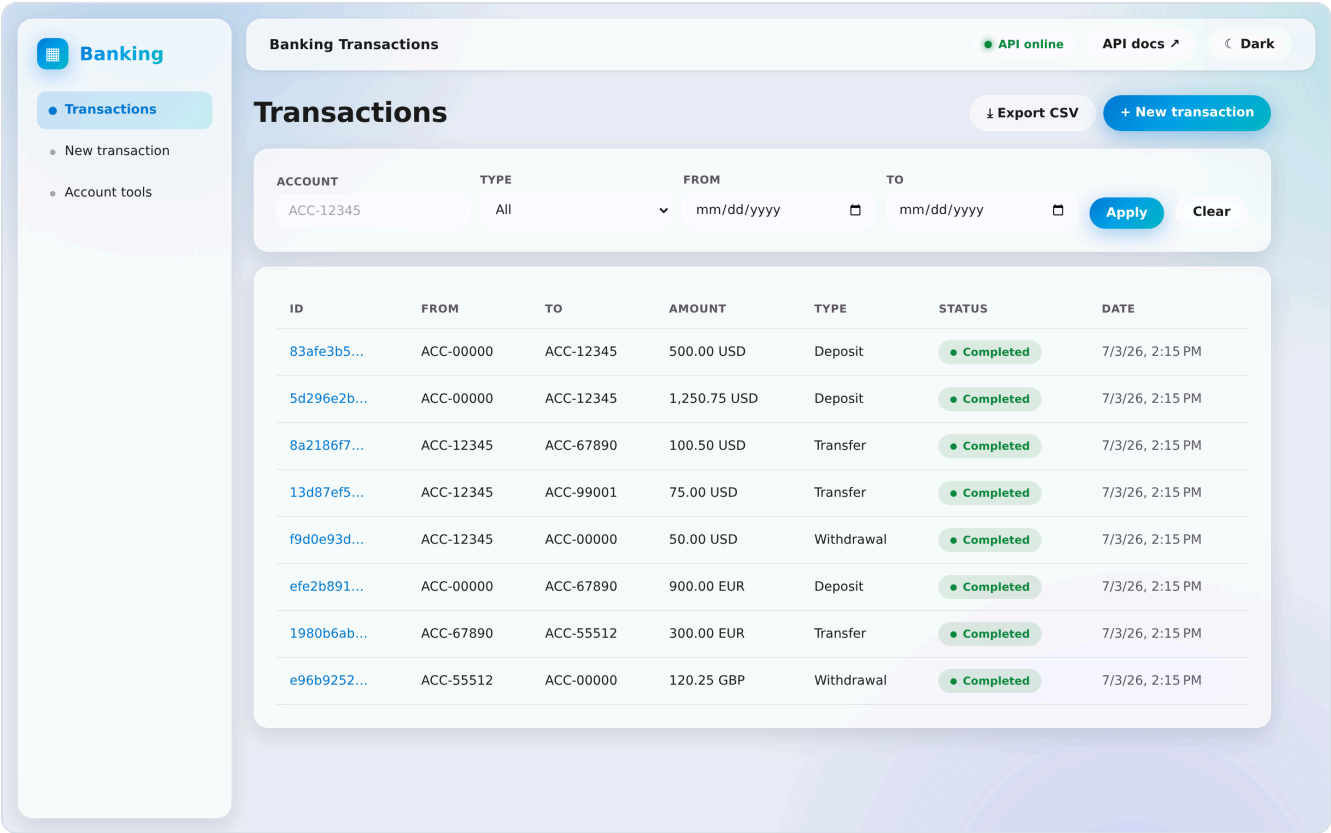
curl -s :3000/transactions | jq length
8 # same contract, same seed data

curl -s :3000/accounts/ACC-12345/balance
{ "accountId": "ACC-12345", "balance": 1525.25 }
```

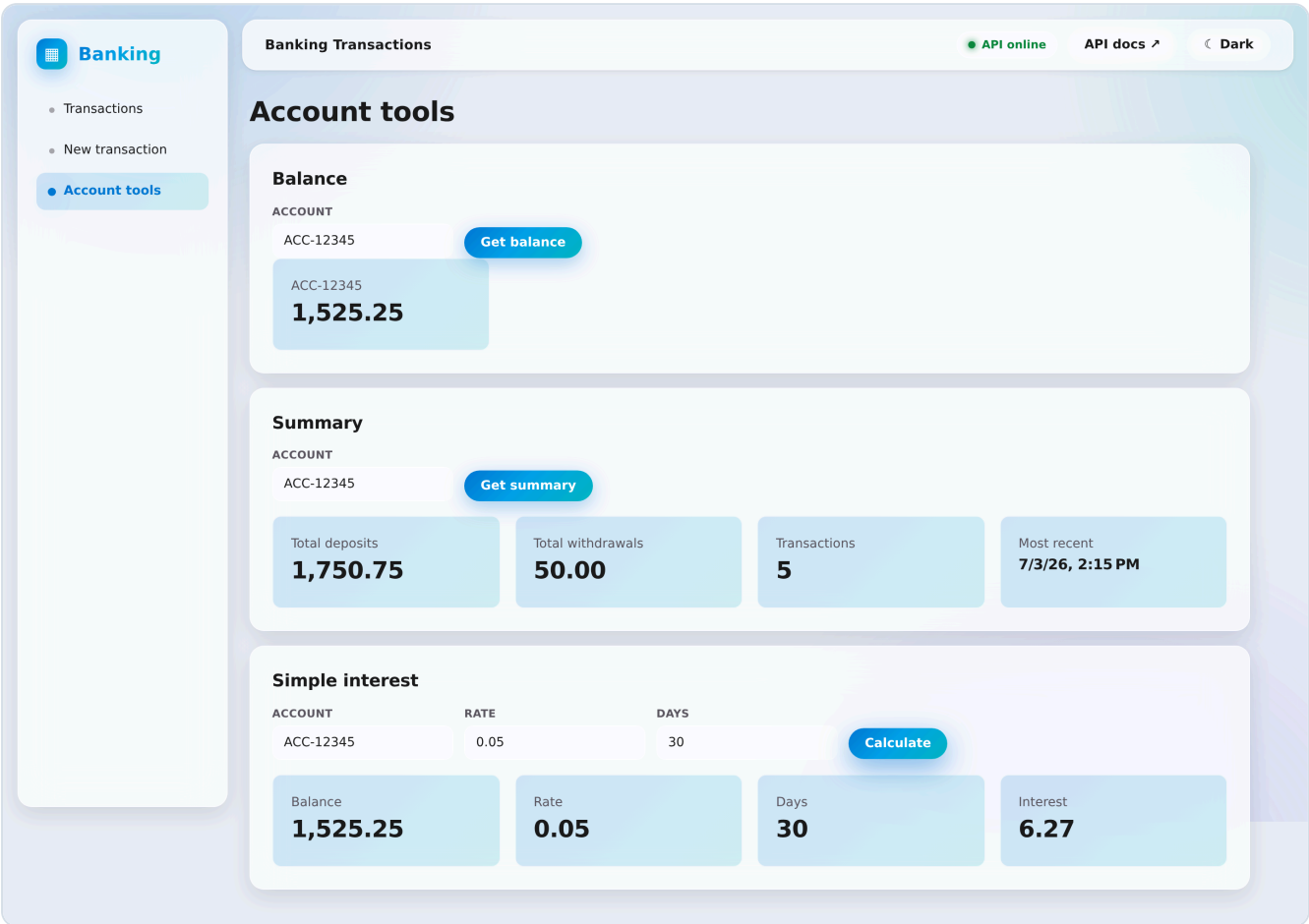
Also available (same contract, same tests): Java + Spring Boot, C# / .NET backends; Flutter Web frontend. Every backend passes the identical TDD suite — see each backend-*/README.md.

Обидва бекенди запущено на :3000; вони віддають однакові засіяні дані.

4. Стек В — Angular (Fluent UI) на бекенді Go



Список транзакцій в Angular — обслуговується бекендом Go.

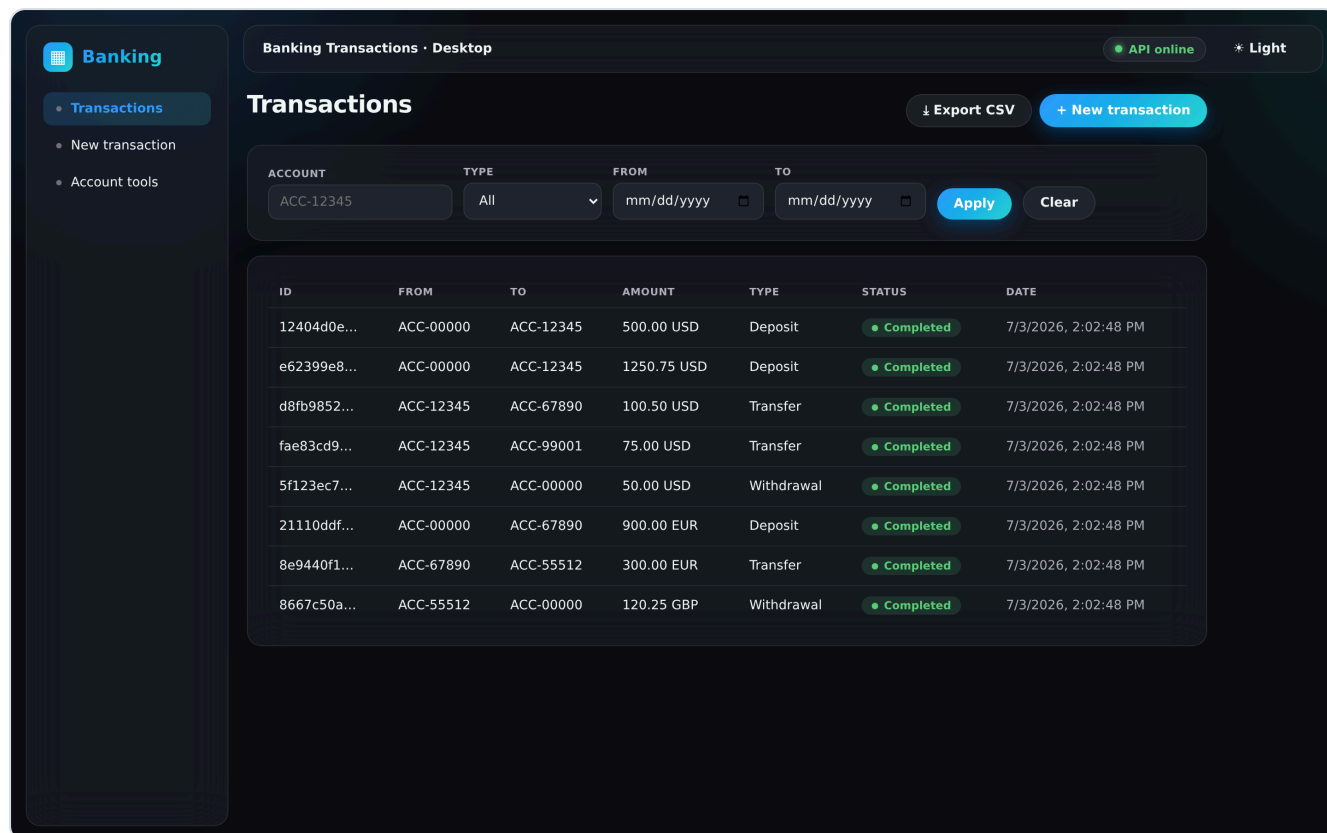


Інструменти рахунку в Angular: баланс, зведення та відсотки, обчислені Go.

5. Перегляд транзакцій

Скріншоти нижче: десктоп-застосунок Electron на бекенді FastAPI.

Вкладка **Transactions** показує кожну транзакцію: короткий ID, рахунки, суму + валюту, тип, кольоровий **бейдж статусу** та дату. Панель фільтрів підтримує **Рахунок** (відправник або отримувач), **Тип** і **діапазон дат** — комбінуються логічним «І». Кнопка **Export CSV** завантажує список.



Список транзакцій із фільтрами та бейджами статусу.

6. Створення транзакції

Натисніть **+ New transaction**, заповніть поля та натисніть **Create transaction**. Форма перевіряє введення й показує повідомлення **під кожним полем**, якщо щось не так.

- **Рахунок «Від / До»** — має відповідати формату `ACC-XXXXXX`.
- **Сума** — додатне число, максимум 2 знаки після коми.
- **Валюта** — валідний код ISO 4217 (USD, EUR, GBP, JPY).
- **Тип** — deposit, withdrawal або transfer.

Banking

• Transactions

New transaction

• Account tools

Banking Transactions · Desktop

API onlineLight

New transaction

FROM ACCOUNT

BADACC

String should match pattern '^ACC-[A-Za-z0-9]+\$'

TO ACCOUNT

ACC-67890

AMOUNT

-10

Input should be greater than 0

CURRENCY

USD

TYPE

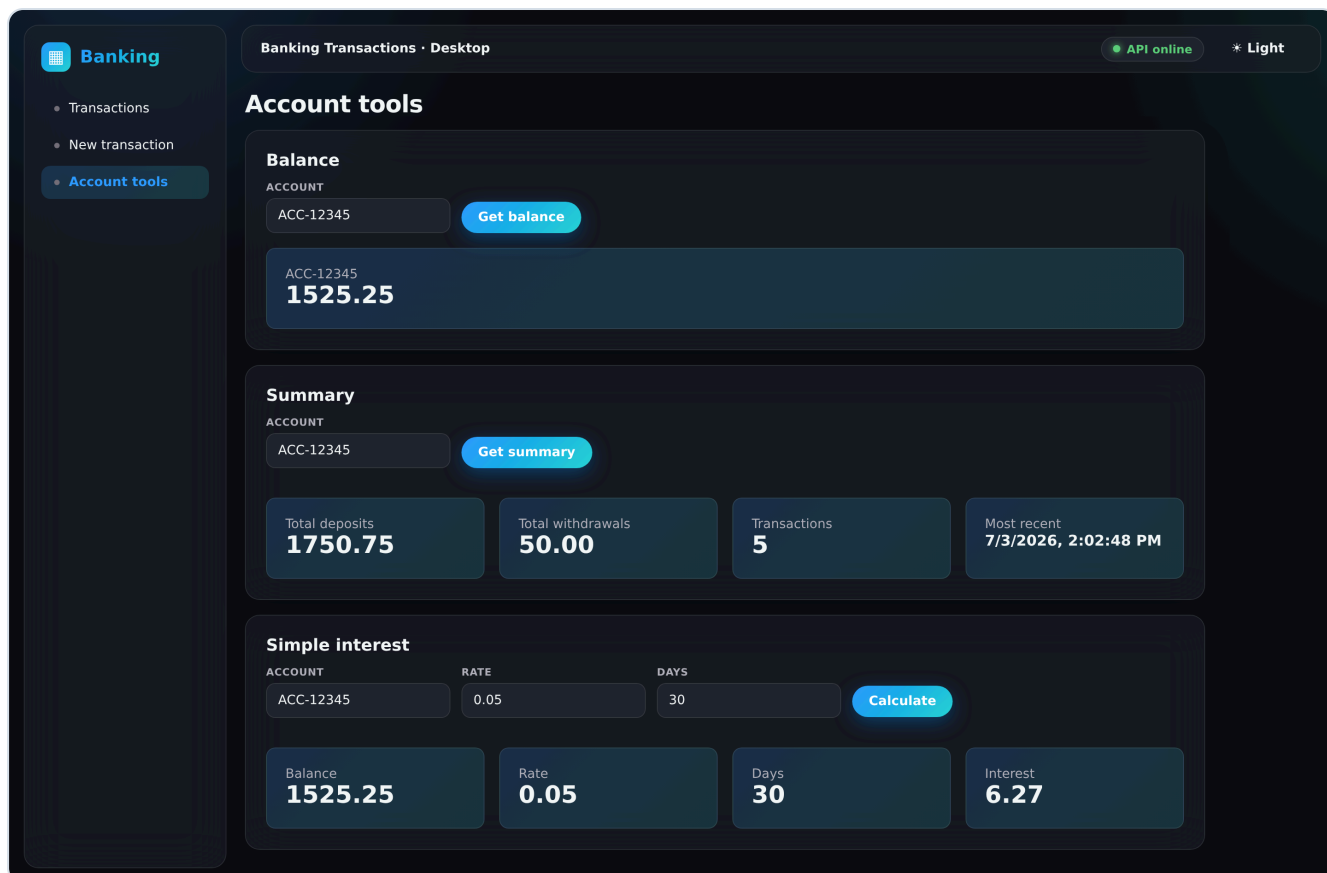
Transfer

Create transactionCancel

Форма нової транзакції з помилками валідації під полями.

7. Інструменти рахунку

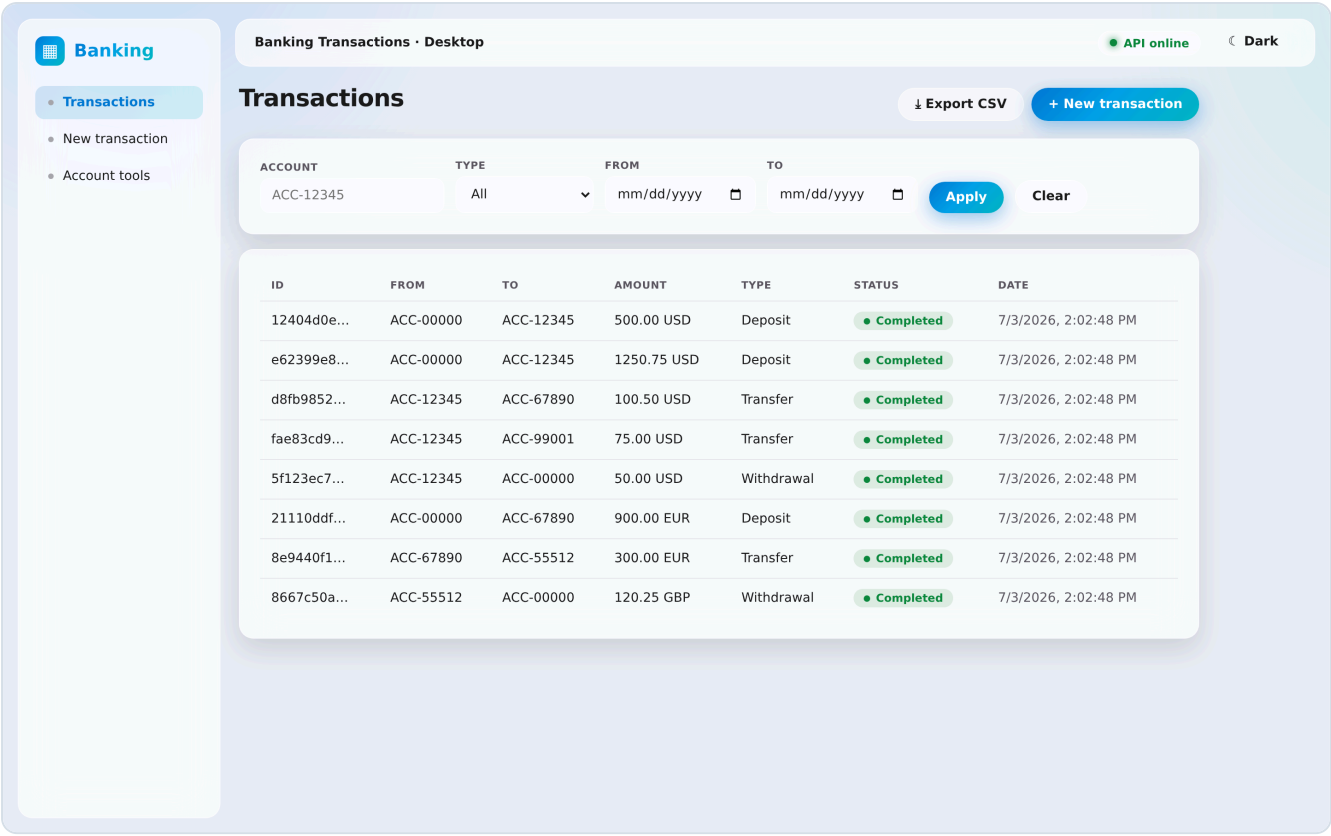
Введіть номер рахунку та натисніть кнопку у відповідній картці: **Баланс** (з виконаних транзакцій), **Зведення** (надходження / зняття / кількість / остання дата) та **Прості відсотки** ($\text{баланс} \times \text{ставка} \times \text{дні} \div 365$, 2 знаки).



Результати: баланс, зведення та прості відсотки.

8. Світла й темна теми

Перемикач * Light / ☾ Dark у правому верхньому куті — таблиці, бейджі й картки адаптуються.



Список транзакцій у світлій темі.

9. REST API

Інтерфейс — це тонкий клієнт над REST API, який можна викликати напряму через `curl` , Postman чи VS Code REST Client. Інтерактивний Swagger UI — за адресою `/docs` , специфікація OpenAPI — `/openapi.json` .

Метод	Ендпоінт	Призначення
POST	<code>/transactions</code>	Створити (201; 400 зі структурованим списком помилок).
GET	<code>/transactions</code>	Список із фільтрами <code>accountId</code> / <code>type</code> / <code>from</code> / <code>to</code> .
GET	<code>/transactions/{id}</code>	Отримати одну (404, якщо немає).
GET	<code>/transactions/export?format=csv</code>	Завантажити все у CSV.
GET	<code>/accounts/{id}/balance</code>	Баланс рахунку.
GET	<code>/accounts/{id}/summary</code>	Надходження / зняття / кількість / дата.
GET	<code>/accounts/{id}/interest?rate=&days=</code>	Прості відсотки.



Banking Transactions API — Reference

Homework 1 · FastAPI implementation · base URL `http://localhost:3000` · live request/response samples

POST /transactions 201 Created

Create a transaction. Server sets id, timestamp, status.

REQUEST

```
curl -X POST http://localhost:3000/transactions \
-H "Content-Type: application/json" \
-d '{"fromAccount": "ACC-12345", "toAccount": "ACC-67890",
  "amount": 100.50, "currency": "USD", "type": "transfer"}'
```

RESPONSE

```
{
  "id": "e8436d57-513b-491f-9e86-e2bc49afd361",
  "fromAccount": "ACC-12345",
  "toAccount": "ACC-67890",
  "amount": 100.5,
  "currency": "USD",
  "type": "transfer",
  "timestamp": "2026-07-03T14:06:32.858715Z",
  "status": "completed"
}
```

GET /accounts/{id}/balance 200 OK

Balance from completed transactions (deposit / withdrawal / transfer rules).

REQUEST

```
curl http://localhost:3000/accounts/ACC-12345/balance
```

RESPONSE

```
{ "accountId": "ACC-12345", "balance": 1424.75 }
```

GET /accounts/{id}/summary 200 OK

Totals + count + most-recent date for an account.

RESPONSE

```
{
  "totalDeposits": 1750.75,
  "totalWithdrawals": 50.0,
  "transactionCount": 6,
  "mostRecentTransactionDate": "2026-07-03T14:06:32Z"
}
```

POST /transactions 400 Bad Request

Invalid input returns a structured error list (bad account format, non-positive amount).

RESPONSE

```
{
  "error": "Validation failed",
  "details": [
    { "field": "fromAccount",
      "message": "account must match ACC-XXXX" },
    { "field": "amount",
      "message": "amount must be a positive number" }
  ]
}
```

GET /accounts/{id}/interest 200 OK

Simple interest = balance × rate × days ÷ 365 (2dp).

REQUEST

```
curl "http://localhost:3000/accounts/ACC-12345/interest?rate=0.05&days=30"
```

RESPONSE

```
{
  "accountId": "ACC-12345", "balance": 1424.75,
  "rate": 0.05, "days": 30, "interest": 5.86
}
```

GET /transactions?type=transfer 200 OK

List with filters: accountId, type, from/to date range (combinable).

RESPONSE (TRUNCATED)

```
[
  {
    "id": "d8fb9852-145a-4f55-...",
    "fromAccount": "ACC-12345",
    "toAccount": "ACC-67890",
    "amount": 100.5,
    "currency": "USD",
    "type": "transfer",
    "status": "completed"
  }, ...
]
```

GET /transactions/export?format=csv 200 OK

Exports all transactions as text/csv (RFC 4180) with a download header.

RESPONSE

```
id,fromAccount,toAccount,amount,currency,type,timestamp,status
d8fb...,ACC-12345,ACC-67890,100.50,USD,transfer,2026-07-03T14:02:48Z,completed
fae8...,ACC-12345,ACC-99001,75.00,USD,transfer,2026-07-03T14:02:48Z,completed
...
```

Also available: interactive Swagger UI at `/docs` and OpenAPI spec at `/openapi.json`.

Ендпоінти REST API з реальними прикладами запитів/відповідей.